

C++ Programming Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This C++ Programming course is designed to build a strong foundation in programming, object-oriented programming, problem-solving, data structures, the Standard Template Library (STL), and modern C++ development.

The course begins with basic syntax and procedural programming and gradually progresses to classes, objects, inheritance, polymorphism, templates, exception handling, file handling, STL, memory management, smart pointers, modern C++ features, multithreading, debugging, testing, and project development.

The syllabus is suitable for beginners, students, engineering graduates, working professionals, and programmers who want to learn C++ for application development, competitive programming, system programming, game development, embedded systems, or Data Structures and Algorithms.

Course Objectives

By the end of this course, learners will be able to:

- Understand the fundamentals of C++ programming.
- Write, compile, execute, and debug C++ programs.
- Use variables, data types, operators, loops, functions, arrays, and strings.
- Apply object-oriented programming principles.
- Design classes and reusable software components.
- Use constructors, destructors, inheritance, polymorphism, and abstraction.
- Understand pointers, references, and memory management.
- Use templates and generic programming.
- Work effectively with the Standard Template Library.
- Use modern C++ features introduced in C++11 and later.
- Handle exceptions and perform file operations.
- Implement common data structures and algorithms.
- Build console-based and modular C++ applications.
- Use Git and GitHub for version control.
- Prepare for C++ programming interviews and coding assessments.

Module 1: Introduction to Programming and C++

Topics

- Introduction to Computer Programming
- What is a Programming Language?
- Low-Level, Middle-Level, and High-Level Languages
- Compiler, Interpreter, Assembler, and Linker
- Source Code, Object Code, and Executable Files
- History and Evolution of C++
- Relationship Between C and C++
- Features of C++
- Applications of C++
- Why Learn C++?
- C++ Standards Overview
- C++98
- C++03
- C++11
- C++14

- C++17
- C++20
- C++23
- Structure of a C++ Program
- Tokens in C++
- Keywords
- Identifiers
- Literals
- Comments
- Header Files
- Namespaces
- `main()` Function
- Writing the First C++ Program
- Compilation and Execution Process

Practical Exercises

- Hello World Program
- Display Personal Information
- Basic Arithmetic Program
- Compile and Run Programs Using GCC / Clang

Module 2: Setting Up the C++ Development Environment

Topics

- Installing a C++ Compiler
- GCC / G++
- Clang
- Microsoft Visual C++ Compiler Overview
- IDEs and Editors
- Visual Studio Code
- Visual Studio
- Code::Blocks
- CLion
- Online Compilers
- Creating and Saving .cpp Files
- Compiling from the Command Line
- Common G++ Commands
- Compilation Stages
- Preprocessing
- Compilation
- Assembly
- Linking
- Syntax Errors
- Compilation Errors
- Linking Errors
- Runtime Errors
- Logical Errors
- Debugging Basics
- Introduction to Build Tools
- Make and CMake Overview

Module 3: Basic Syntax, Variables, Constants, and Data Types

Topics

- Variables in C++
- Variable Declaration
- Variable Initialization
- Uniform Initialization
- Constants
- const Keyword
- constexpr Overview
- Fundamental Data Types
- bool
- char
- wchar_t
- int
- float
- double
- void
- Type Modifiers
- short
- long
- signed
- unsigned
- Fixed-Width Integer Types
- auto Type Deduction
- decltype
- sizeof
- Type Ranges
- Type Conversion
- Implicit Conversion
- Explicit Conversion
- C-Style Casts
- C++ Cast Operators Overview
- static_cast
- dynamic_cast
- const_cast
- reinterpret_cast
- Scope
- Lifetime
- Escape Sequences

Practical Exercises

- Area and Perimeter Calculator
- Temperature Conversion
- Simple Interest Calculator
- Swap Two Numbers
- Display Sizes of Data Types

Module 4: Input and Output

Topics

- Standard Input and Output
- `std::cout`
- `std::cin`
- `std::cerr`

- `std::clog`
- Stream Insertion Operator `<<`
- Stream Extraction Operator `>>`
- `std::endl`
- Formatting Output
- `<iomanip>`
- `setw`
- `setprecision`
- `fixed`
- `scientific`
- Character Input
- String Input
- `getline()`
- Input Buffer Issues
- Input Validation

Practical Exercises

- Read and Display Student Details
- Arithmetic Calculator
- Formatted Invoice
- Read Full Name and Address
- Numeric Input Validation

Module 5: Operators and Expressions

Topics

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Increment and Decrement Operators
- Bitwise Operators
- AND
- OR
- XOR
- NOT
- Left Shift
- Right Shift
- Conditional / Ternary Operator
- `sizeof`
- Scope Resolution Operator `::`
- Member Access Operators
- Operator Precedence
- Operator Associativity
- Expressions
- Lvalues and Rvalues
- Short-Circuit Evaluation
- Type Conversion in Expressions

Practical Exercises

- Even or Odd Number
- Maximum of Two Numbers
- Maximum of Three Numbers

- Leap Year
- Bitwise Operations

Module 6: Decision-Making Statements

Topics

- Control Flow
- if
- if-else
- Nested if
- else-if Ladder
- switch
- case
- default
- break
- Nested switch
- Conditional Operator
- Best Practices for Decision Logic

Practical Exercises

- Grade Calculator
- Positive, Negative, or Zero
- Vowel or Consonant
- Menu-Driven Calculator
- Electricity Bill Calculator

Module 7: Looping Statements

Topics

- while
- do-while
- for
- Range-Based for
- Nested Loops
- Infinite Loops
- break
- continue
- goto Overview
- Loop Design Best Practices

Practical Exercises

- Multiplication Table
- Sum of Natural Numbers
- Factorial
- Fibonacci Series
- Reverse a Number
- Palindrome Number
- Armstrong Number
- Prime Number
- Prime Numbers in a Range
- Number Patterns
- Star Patterns

Module 8: Functions

Topics

- Introduction to Functions
- Function Declaration
- Function Definition
- Function Call
- Function Prototypes
- Return Types
- Parameters and Arguments
- Pass by Value
- Pass by Reference
- Pass by Pointer
- Default Arguments
- Inline Functions
- Function Overloading
- Recursive Functions
- Local and Global Variables
- Static Local Variables
- Function Scope
- Namespace Scope
- Lambda Functions Introduction

Practical Exercises

- Function-Based Calculator
- Factorial Using Recursion
- Fibonacci Using Recursion
- GCD and LCM
- Swap Using References
- Overloaded Area Functions

Module 9: Arrays and Multidimensional Arrays

Topics

- Introduction to Arrays
- One-Dimensional Arrays
- Declaration
- Initialization
- Traversal
- Passing Arrays to Functions
- Array Size
- Searching
- Sorting
- Two-Dimensional Arrays
- Matrices
- Multidimensional Arrays
- C-Style Arrays vs `std::array`
- Range-Based Loops with Arrays

Practical Exercises

- Sum and Average
- Maximum and Minimum
- Reverse an Array

- Linear Search
- Binary Search
- Bubble Sort
- Selection Sort
- Insertion Sort
- Matrix Addition
- Matrix Multiplication
- Matrix Transpose

Module 10: Strings

Topics

- C-Style Strings
- Character Arrays
- Null Character
- String Literals
- <cstring> Functions
- Introduction to std::string
- String Declaration and Initialization
- String Input and Output
- String Concatenation
- String Comparison
- String Length
- Substrings
- Find and Replace
- Insert and Erase
- String Iteration
- Conversion Between Numbers and Strings
- std::stoi
- std::stol
- std::stof
- std::stod
- std::to_string
- String Streams
- std::stringstream

Practical Exercises

- Reverse a String
- Check Palindrome
- Count Vowels and Consonants
- Count Words
- Character Frequency
- Remove Spaces
- Find Substring
- Convert String to Number

Module 11: Pointers and References

Topics

- Memory Addresses
- Pointer Variables
- Address Operator &
- Dereference Operator *

- Pointer Initialization
- Null Pointers
- `nullptr`
- Pointer Arithmetic
- Pointers and Arrays
- Pointers and Strings
- Pointer to Pointer
- Passing Pointers to Functions
- References
- Reference Variables
- Pointers vs References
- Const Pointers
- Pointer to Const
- Dangling Pointers
- Wild Pointers
- Function Pointers
- Pointer-to-Member Overview

Practical Exercises

- Swap Using Pointers
- Traverse Array Using Pointers
- Dynamic Array Access
- Pointer Arithmetic Demonstrations
- Function Pointer Example

Module 12: Dynamic Memory Management

Topics

- Stack vs Heap
- Dynamic Memory Allocation
- `new`
- `delete`
- Dynamic Arrays
- `new[]`
- `delete[]`
- Memory Leaks
- Dangling Pointers
- Double Deletion
- Resource Management
- RAII Introduction
- C-Style Allocation vs C++ Allocation
- `malloc/free` vs `new/delete`
- Smart Pointer Introduction

Practical Exercises

- Dynamic Integer Allocation
- Dynamic Array
- Dynamic Student Records
- Dynamic Matrix
- Memory Leak Demonstrations and Fixes

Module 13: Introduction to Object-Oriented Programming

Topics

- Procedural Programming vs Object-Oriented Programming
- Object-Oriented Programming Concepts
- Class
- Object
- Encapsulation
- Abstraction
- Inheritance
- Polymorphism
- Benefits of OOP
- Real-World Modeling
- Defining Classes
- Creating Objects
- Data Members
- Member Functions
- Access Specifiers
- public
- private
- protected
- Class Scope
- Object Memory

Practical Exercises

- Student Class
- Employee Class
- Bank Account Class
- Product Class

Module 14: Constructors and Destructors

Topics

- Constructors
- Default Constructor
- Parameterized Constructor
- Constructor Overloading
- Copy Constructor
- Move Constructor Overview
- Delegating Constructors
- Constructor Initialization Lists
- Destructors
- Constructor and Destructor Execution Order
- Object Lifetime
- Shallow Copy
- Deep Copy
- Rule of Three
- Rule of Five
- Rule of Zero

Practical Exercises

- Constructor Overloading
- Copy Constructor Example
- Deep Copy Class
- Resource-Managing Class

Module 15: Encapsulation and Abstraction

Topics

- Data Hiding
- Encapsulation
- Getters and Setters
- Validation Through Member Functions
- Abstraction
- Interface Design
- Separating Interface and Implementation
- Header and Source Files
- Const Member Functions
- mutable Keyword Overview
- this Pointer

Practical Exercises

- Secure Bank Account Class
- Employee Salary Class
- Encapsulated Inventory Class

Module 16: Inheritance

Topics

- Introduction to Inheritance
- Base Class
- Derived Class
- Types of Inheritance
- Single
- Multilevel
- Multiple
- Hierarchical
- Hybrid
- Access Modes
- Public Inheritance
- Protected Inheritance
- Private Inheritance
- Constructor and Destructor Order
- Function Redefinition
- Name Hiding
- Base Class Initialization
- Multiple Inheritance
- Ambiguity in Multiple Inheritance
- Diamond Problem
- Virtual Base Classes

Practical Exercises

- Person and Student
- Employee Hierarchy
- Shape Hierarchy
- Multiple Inheritance Demonstration

Module 17: Polymorphism

Topics

- Introduction to Polymorphism
- Compile-Time Polymorphism
- Function Overloading
- Operator Overloading
- Runtime Polymorphism
- Function Overriding
- Virtual Functions
- Virtual Function Table Concept
- Pure Virtual Functions
- Abstract Classes
- Interfaces in C++
- Virtual Destructors
- `override`
- `final`
- Object Slicing
- Dynamic Binding

Practical Exercises

- Shape Area Using Virtual Functions
- Employee Salary Calculation
- Payment Method Interface
- Abstract Vehicle Hierarchy

Module 18: Operator Overloading

Topics

- Introduction to Operator Overloading
- Rules and Restrictions
- Unary Operator Overloading
- Binary Operator Overloading
- Member vs Non-Member Overloads
- Friend Functions
- Stream Operator Overloading
- `<<`
- `>>`
- Comparison Operators
- Arithmetic Operators
- Increment and Decrement Operators
- Subscript Operator
- Function Call Operator
- Modern Comparison with `<=>` Overview

Practical Exercises

- Complex Number Operations
- Matrix Addition
- Custom String Class
- Stream Output for User-Defined Types

Module 19: Friend Functions and Friend Classes

Topics

- Friend Function
- Friend Class

- Accessing Private Members
- Use Cases
- Advantages and Risks
- Encapsulation Considerations

Practical Exercises

- Complex Number Friend Function
- Two-Class Data Sharing Example

Module 20: Static Members and Const Correctness

Topics

- Static Data Members
- Static Member Functions
- Class-Level Data
- Static Local Variables
- Const Objects
- Const Member Functions
- Const References
- Const Pointers
- constexpr
- constexpr Overview
- constexpr Overview
- Const Correctness Best Practices

Module 21: Templates and Generic Programming

Topics

- Introduction to Templates
- Function Templates
- Class Templates
- Multiple Template Parameters
- Template Specialization
- Full Specialization
- Partial Specialization Overview
- Non-Type Template Parameters
- Variadic Templates Overview
- Generic Programming
- Template Type Deduction
- Concepts Overview
- Constraints Overview

Practical Exercises

- Generic Maximum Function
- Generic Swap
- Generic Stack Class
- Generic Pair-Like Class

Module 22: Exception Handling

Topics

- Introduction to Exceptions
- try

- throw
- catch
- Multiple Catch Blocks
- Catch-All Handler
- Standard Exceptions
- Custom Exception Classes
- Exception Propagation
- Stack Unwinding
- noexcept
- Exception Safety
- RAII and Exception Handling
- When Not to Use Exceptions

Practical Exercises

- Division-by-Zero Handling
- Invalid Age Exception
- Bank Withdrawal Exception
- File Open Error Handling

Module 23: File Handling

Topics

- Streams in C++
- <fstream>
- ifstream
- ofstream
- fstream
- Opening Files
- Closing Files
- File Modes
- Text Files
- Binary Files
- Reading Files
- Writing Files
- Appending Files
- File Positioning
- seekg
- seekp
- tellg
- tellp
- Serializing Simple Objects
- File Error Handling
- CSV File Processing Overview

Practical Exercises

- Write Student Data to File
- Read Student Data from File
- Copy a File
- Word and Line Counter
- Employee Record System
- Binary Record Storage

Module 24: Standard Template Library (STL) Introduction

Topics

- What is STL?
- Components of STL
- Containers
- Iterators
- Algorithms
- Function Objects
- Generic Programming with STL
- Benefits of STL
- Choosing Appropriate Containers

Module 25: STL Sequence Containers**Topics**

- `std::vector`
- `std::array`
- `std::deque`
- `std::list`
- `std::forward_list`
- Container Operations
- Iteration
- Capacity
- Insertion
- Deletion
- Searching
- Performance Characteristics

Practical Exercises

- Dynamic Student List Using Vector
- Queue-Like Operations with Deque
- Linked List Operations with `std::list`

Module 26: STL Associative Containers**Topics**

- `std::set`
- `std::multiset`
- `std::map`
- `std::multimap`
- Ordered Containers
- Keys and Values
- Searching
- Insertion
- Deletion
- Iteration
- Custom Comparators

Practical Exercises

- Word Frequency Using Map
- Unique Elements Using Set
- Student Marks Database Using Map

Module 27: STL Unordered Containers

Topics

- Hashing Concepts
- `std::unordered_set`
- `std::unordered_multiset`
- `std::unordered_map`
- `std::unordered_multimap`
- Hash Functions
- Bucket Concepts
- Ordered vs Unordered Containers
- Performance Considerations

Practical Exercises

- Fast Frequency Counter
- Lookup Table
- Duplicate Detection

Module 28: STL Container Adapters**Topics**

- `std::stack`
- `std::queue`
- `std::priority_queue`
- Use Cases
- Underlying Containers

Practical Exercises

- Parentheses Checker Using Stack
- Task Queue
- Priority-Based Job Scheduler

Module 29: Iterators**Topics**

- Introduction to Iterators
- Iterator Categories
- Input Iterator
- Output Iterator
- Forward Iterator
- Bidirectional Iterator
- Random Access Iterator
- Contiguous Iterator Overview
- `begin()`
- `end()`
- `cbegin()`
- `cend()`
- Reverse Iterators
- Iterator Invalidation
- Range-Based Loops

Module 30: STL Algorithms**Topics**

- `<algorithm>`

- Searching Algorithms
- Sorting Algorithms
- Counting Algorithms
- Transformation Algorithms
- `std::sort`
- `std::stable_sort`
- `std::find`
- `std::find_if`
- `std::count`
- `std::count_if`
- `std::reverse`
- `std::rotate`
- `std::transform`
- `std::remove`
- `std::unique`
- `std::min_element`
- `std::max_element`
- `std::binary_search`
- `std::lower_bound`
- `std::upper_bound`
- `std::accumulate`
- Predicates
- Lambdas with Algorithms

Practical Exercises

- Sort Student Records
- Search Custom Objects
- Remove Duplicate Values
- Transform Numeric Data
- Filter Values Using Lambdas

Module 31: Lambda Expressions

Topics

- Introduction to Lambdas
- Lambda Syntax
- Capture Lists
- Capture by Value
- Capture by Reference
- Parameter Lists
- Return Types
- Mutable Lambdas
- Generic Lambdas
- Lambdas with STL Algorithms
- Immediately Invoked Lambdas

Practical Exercises

- Sort with Custom Lambda
- Filter a Vector
- Calculate Values Using Captured Variables

Module 32: Smart Pointers and RAI

Topics

- Problems with Raw Pointers
- Resource Acquisition Is Initialization
- `std::unique_ptr`
- `std::shared_ptr`
- `std::weak_ptr`
- Ownership Models
- `std::make_unique`
- `std::make_shared`
- Reference Counting
- Circular References
- Custom Deleters Overview
- Smart Pointers vs Raw Pointers

Practical Exercises

- Dynamic Object with `unique_ptr`
- Shared Resource with `shared_ptr`
- Breaking Circular References with `weak_ptr`

Module 33: Move Semantics and Perfect Forwarding

Topics

- Lvalues
- Rvalues
- Lvalue References
- Rvalue References
- Move Constructor
- Move Assignment Operator
- `std::move`
- Copy vs Move
- Rule of Five
- Perfect Forwarding Overview
- `std::forward`
- Performance Benefits
- Moved-From Objects

Practical Exercises

- Implement Move Constructor
- Compare Copy and Move Operations
- Resource-Managing Class

Module 34: Modern C++ Features

C++11 Features

- `auto`
- `nullptr`
- Range-Based `for`
- Lambda Expressions
- Smart Pointers
- Move Semantics
- `enum class`
- `constexpr`
- Uniform Initialization
- Variadic Templates

- `override`
- `final`

C++14 Features

- Generic Lambdas
- Return Type Deduction
- `std::make_unique`
- Binary Literals
- Digit Separators

C++17 Features

- Structured Bindings
- `if constexpr`
- `std::optional`
- `std::variant`
- `std::any`
- `std::string_view`
- Fold Expressions
- Filesystem Library
- Parallel Algorithms Overview

C++20 Features

- Concepts
- Ranges
- Coroutines Overview
- Modules Overview
- Three-Way Comparison Operator
- `std::span`
- `constexpr`
- `constexpr`
- Calendar and Timezone Enhancements

C++23 Overview

- Standard Library Enhancements
- `std::expected`
- Improved Ranges
- `std::print` Overview
- Modern Language Improvements

Module 35: Namespaces and Modular Code Organization

Topics

- Namespaces
- Named Namespaces
- Nested Namespaces
- Anonymous Namespaces
- Namespace Aliases
- `using` Declarations
- `using namespace`
- Header Files
- Source Files
- Include Guards
- `#pragma once`
- Forward Declarations

- Separate Compilation
- Project Folder Structure
- C++ Modules Overview

Module 36: Preprocessor Directives and Macros

Topics

- C++ Preprocessor
- `#include`
- `#define`
- `#undef`
- Function-Like Macros
- Conditional Compilation
- `#if`
- `#ifdef`
- `#ifndef`
- `#else`
- `#elif`
- `#endif`
- Predefined Macros
- Header Guards
- Macro Pitfalls
- `constexpr` and Inline Functions as Alternatives

Module 37: Type Casting and Runtime Type Information

Topics

- Implicit Casting
- Explicit Casting
- `static_cast`
- `dynamic_cast`
- `const_cast`
- `reinterpret_cast`
- C-Style Casting
- RTTI
- `typeid`
- Polymorphic Types
- Safe Casting Practices

Module 38: Advanced Object-Oriented Design Concepts

Topics

- Composition
- Aggregation
- Association
- Inheritance vs Composition
- Dependency
- Interface Segregation
- Coupling and Cohesion
- SOLID Principles Overview
- Single Responsibility Principle
- Open/Closed Principle
- Liskov Substitution Principle

- Interface Segregation Principle
- Dependency Inversion Principle
- Basic Design Patterns
- Singleton
- Factory
- Strategy
- Observer
- Adapter

Module 39: Data Structures Using C++

Arrays and Dynamic Arrays

- Static Arrays
- Dynamic Arrays
- `std::vector`

Linked Lists

- Singly Linked List
- Doubly Linked List
- Circular Linked List
- Insertion
- Deletion
- Traversal
- Searching
- Reversal

Stacks

- Array-Based Stack
- Linked-List-Based Stack
- STL Stack

Queues

- Simple Queue
- Circular Queue
- Priority Queue
- STL Queue

Trees

- Binary Tree
- Binary Search Tree
- Tree Traversals
- Preorder
- Inorder
- Postorder
- Level Order

Graphs Overview

- Graph Representation
- Adjacency Matrix
- Adjacency List
- Breadth-First Search
- Depth-First Search

Hashing

- Hash Tables
- Collision Concepts
- STL Unordered Containers

Module 40: Algorithms and Complexity

Topics

- Algorithm Design
- Time Complexity
- Space Complexity
- Big-O Notation
- Best, Average, and Worst Case
- Searching Algorithms
- Linear Search
- Binary Search
- Sorting Algorithms
- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Heap Sort Overview
- Recursion
- Backtracking Overview
- Greedy Algorithms Overview
- Dynamic Programming Introduction

Module 41: Multithreading and Concurrency

Topics

- Introduction to Concurrency
- Processes vs Threads
- `std::thread`
- Creating Threads
- Joining Threads
- Detaching Threads
- Race Conditions
- Critical Sections
- Mutexes
- `std::mutex`
- `std::lock_guard`
- `std::unique_lock`
- Deadlocks
- Condition Variables
- `std::condition_variable`
- Atomic Operations
- `std::atomic`
- Futures and Promises
- `std::async`
- Thread Safety Basics

Practical Exercises

- Run Multiple Tasks in Threads

- Thread-Safe Counter
- Producer-Consumer Example

Module 42: File System and Utility Libraries

Topics

- `<filesystem>`
- Paths
- Files
- Directories
- Directory Iteration
- File Status
- Creating and Removing Directories
- `std::chrono`
- Time Measurement
- Random Number Library
- Regular Expressions Overview
- `std::regex`

Module 43: Debugging and Error Diagnosis

Topics

- Types of Programming Errors
- Compiler Warnings
- Debugging Techniques
- Print Debugging
- GDB Basics
- Visual Studio Debugger Overview
- Breakpoints
- Step Into
- Step Over
- Watch Variables
- Call Stack
- Assertions
- `assert`
- Core Dumps Overview
- AddressSanitizer
- UndefinedBehaviorSanitizer
- MemorySanitizer Overview
- Valgrind Overview

Module 44: Testing in C++

Topics

- Importance of Testing
- Unit Testing Concepts
- Test Cases
- Assertions
- Boundary Testing
- Positive and Negative Testing
- Regression Testing
- Introduction to Testing Frameworks
- GoogleTest

- Catch2
- Testable Code Design
- Mocking Overview
- Test-Driven Development Introduction

Module 45: Build Systems and Dependency Management

Topics

- Compilation and Linking Review
- Make
- Makefiles
- CMake
- CMakeLists.txt
- Debug and Release Builds
- Static Libraries
- Dynamic Libraries
- Linking Libraries
- Package Management Overview
- Conan Overview
- vcpkg Overview

Module 46: Secure and Professional C++ Programming Practices

Topics

- Coding Standards
- Naming Conventions
- Readable Code
- Modular Design
- Avoiding Magic Numbers
- Const Correctness
- Resource Management
- RAI
- Smart Pointer Usage
- Exception Safety
- Bounds Safety
- Avoiding Undefined Behavior
- Avoiding Memory Leaks
- Avoiding Dangling References
- Avoiding Raw Ownership
- Compiler Warning Flags
- Static Analysis
- Sanitizers
- Code Reviews
- C++ Core Guidelines Overview

Module 47: Problem-Solving and Competitive Programming Foundations

Topics

- Understanding Problem Statements
- Identifying Inputs and Outputs
- Constraints Analysis
- Pseudocode
- Dry Runs

- Edge Cases
- Time and Space Complexity
- Choosing Appropriate Data Structures
- STL for Competitive Programming
- Fast Input/Output Basics
- Common Coding Patterns
- Two Pointers
- Sliding Window
- Prefix Sum
- Hashing
- Binary Search
- Recursion
- Backtracking

Practice Categories

- Number Problems
- Pattern Problems
- Array Problems
- String Problems
- Matrix Problems
- STL Problems
- OOP Problems
- Recursion Problems
- Data Structure Problems

Module 48: Git and GitHub for C++ Developers

Topics

- Introduction to Version Control
- Installing Git
- Creating a Repository
- `git init`
- `git add`
- `git commit`
- `git status`
- `git log`
- Branching
- Merging
- GitHub Repository Creation
- Push and Pull
- Pull Requests Overview
- README Documentation
- `.gitignore`
- Managing Build Files
- Building a Programming Portfolio

Module 49: AI-Assisted C++ Development

Topics

- Using AI Coding Assistants Responsibly
- Generating Boilerplate Code
- Explaining Existing C++ Code
- Debugging with AI Assistance

- Understanding Compiler Errors
- Generating Unit Tests
- Refactoring Suggestions
- Modernizing Legacy C++ Code
- Reviewing STL Usage
- Reviewing Memory Management
- Detecting Potential Bugs
- Security Risks of Blindly Using Generated Code
- Validating AI-Generated Code
- Prompting Techniques for Programming Help

Recommended Practice

Students should learn to:

- Ask AI for explanations instead of only final solutions.
- Review generated code manually.
- Compile with warnings enabled.
- Use sanitizers and tests.
- Test edge cases.
- Understand ownership and lifetime.
- Verify the complexity of generated algorithms.
- Understand every generated line before submission.

Module 50: Mini Projects

Students should complete multiple projects such as:

- Calculator Application
- Student Management System
- Library Management System
- Employee Management System
- Banking Application
- Inventory Management System
- Contact Management System
- Hotel Reservation System
- Quiz Application
- Tic-Tac-Toe Game
- Expense Tracker
- Task Management Application
- Vehicle Rental System
- File-Based Address Book
- E-Commerce Cart Simulation
- Shape Management System Using Polymorphism
- STL-Based Student Database

Module 51: Capstone Project

Learners should build one complete object-oriented C++ project.

Suggested Capstone Projects

- Library Management System
- Hospital Management System
- Banking Management System
- Employee Payroll System
- Inventory and Billing System

- Hotel Reservation System
- Student Information System
- Vehicle Rental System
- E-Commerce Console Application
- Task and Project Management System

Capstone Requirements

The project should demonstrate:

- Object-oriented design
- Classes and objects
- Encapsulation
- Inheritance where appropriate
- Polymorphism where appropriate
- Constructors and destructors
- STL containers
- Algorithms
- Exception handling
- File handling
- Smart pointers where appropriate
- Modular source files
- Input validation
- Error handling
- Unit testing basics
- Git-based version control
- README documentation

Module 52: Interview Preparation

Core C++ Interview Topics

- C++ vs C
- Variables and Data Types
- Pointers and References
- Stack vs Heap
- new and delete
- Classes and Objects
- Constructors and Destructors
- Copy Constructor
- Move Constructor
- Shallow vs Deep Copy
- Inheritance
- Polymorphism
- Virtual Functions
- Virtual Destructors
- Abstract Classes
- Operator Overloading
- Templates
- Exception Handling
- STL Containers
- Iterators
- Algorithms
- Smart Pointers
- RAI
- Lambda Expressions

- `auto`
- `constexpr`
- Move Semantics
- Rule of Three / Five / Zero
- `static`
- `const`
- `mutable`
- Function Overloading vs Overriding
- Object Slicing
- Memory Leaks
- Undefined Behavior
- Multithreading Basics

Interview Activities

- Conceptual Questions
- Output-Based Questions
- Debugging Questions
- Coding Challenges
- STL-Based Problems
- OOP Design Questions
- Data Structure Problems
- Mock Interviews
- Timed Coding Tests

Recommended Lab Assignments

Learners should complete at least:

- 20 Basic Syntax and Operator Programs
- 20 Conditional and Looping Programs
- 15 Function and Recursion Programs
- 20 Array and String Programs
- 15 Pointer and Reference Programs
- 20 OOP Programs
- 10 Inheritance and Polymorphism Programs
- 10 Template and Exception Programs
- 20 STL Exercises
- 10 File Handling Programs
- 15 Data Structure Problems
- 10 Algorithm Problems
- 5 Mini Projects
- 1 Capstone Project

Suggested Course Duration

- Section — Suggested Duration
- Programming Fundamentals and C++ Basics — 10–12 Hours
- Control Flow, Functions, Arrays, and Strings — 15–18 Hours
- Pointers, References, and Memory — 10–12 Hours
- Object-Oriented Programming — 20–25 Hours
- Templates, Exceptions, and File Handling — 10–12 Hours
- STL and Generic Programming — 18–22 Hours
- Modern C++ — 12–15 Hours
- Data Structures and Algorithms — 20–25 Hours
- Concurrency, Testing, and Build Tools — 10–15 Hours

- Projects and Interview Preparation — 20–25 Hours

Recommended Total Duration: approximately **145–180 hours**, depending on the depth of labs, DSA coverage, and project work.

Prerequisites

No prior programming experience is required.

Basic familiarity with computers, files, folders, and command-line usage is helpful.

Learners who already know C programming may progress faster through the introductory modules.

Tools and Technologies

- C++ Programming Language
- GCC / G++
- Clang
- Microsoft Visual C++ Compiler
- Visual Studio Code
- Visual Studio
- CLion
- GDB
- Git
- GitHub
- Make
- CMake
- GoogleTest / Catch2
- Valgrind
- AddressSanitizer
- Linux / macOS / Windows Command Line

Learning Outcomes

After completing the course, learners should be able to:

- Write structured, modular, and object-oriented C++ applications.
- Apply encapsulation, abstraction, inheritance, and polymorphism.
- Use pointers, references, dynamic memory, and smart pointers safely.
- Design reusable classes and generic components using templates.
- Use STL containers, iterators, and algorithms effectively.
- Apply modern C++ features in practical applications.
- Handle exceptions and files.
- Implement common data structures and algorithms.
- Build and organize multi-file C++ projects.
- Debug and test applications.
- Use Git and GitHub professionally.
- Solve C++ coding and interview problems.
- Progress toward system programming, game development, embedded systems, competitive programming, or advanced software engineering.

Recommended Progression After C++

After completing C++, learners can continue with:

- Data Structures and Algorithms
- Advanced Modern C++

- Competitive Programming
- System Programming
- Operating Systems
- Linux Programming
- Embedded Systems
- Game Development
- Unreal Engine
- Computer Graphics
- Networking
- Design Patterns
- High-Performance Computing
- Parallel and Concurrent Programming
- Low-Latency Application Development

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.